

1 - Date

programmation objet, constructeur, accesseurs, str()

On veut définir une classe `Date` dont les attributs sont le jour, le mois et l'année (entiers).

```
class Date :
    def __init__(self, j :int, m:int, a:int): #...
    def get_jour(self): #...
    def set_jour(self, j:int) : #
    def get_mois(self): #...
    def set_mois(self, m:int): #...
    def get_annee(self): #...
    def set_annee(self, a:int): #...
    def lendemain(self) : # renvoie la date correspondant au lendemain de self
    def __str__(self) : #...
```

1. Implémenter son constructeur `__init__()` et les getter et setter `get_...()`, `set_...()` en vérifiant la cohérence des paramètres.
2. Écrire la méthode `__str__()` qui retourne la chaîne de caractère avec la date au format suivant : `jj/mm/aaaa`.
3. Ajouter la méthode `lendemain()` qui calcule et retourne la date du lendemain d'un objet `Date` (on considérera que l'année n'est pas bissextile).
4. Écrire le programme principal qui affichera toutes les dates de l'année 2026 sous la forme suivante (une ligne par mois):

```
01/01/2026 02/01/2026 ...                ... 31/01/2026
01/02/2026 ...                ... 28/02/2026
01/03/2026 ...
...
01/12/2026 ...                ... 31/12/2026
```

2 - Généalogie

programmation objet, encapsulation properties, surcharge d'opérateurs, héritage

Dans l'objectif de développer une application de généalogie, nous allons définir quelques composants du programme. Une instance de la classe `Famille` gère un ensemble d'individus et un ensemble de liens entre les individus (mariage, filiation, etc...).

2.1 Modifier la classe Date

Ajouter à la classe `Date` de l'exercice précédent une méthode `__lt__(self, other)` qui surcharge l'opérateur de comparaison `<`: renvoie `True` si `self` est antérieure à `other`, `False` sinon

2.2 Définir la classe `Individu`

Un objet de la classe `Individu` est caractérisé par les attributs `_nom` et `_prenoms` de type `str` et `_naissance` et `_mort` de type `Date` représentant sa date de naissance et sa date de décès le cas échéant. Si l'individu est vivant, cet attribut `_mort` est égal à la valeur `None`. Le constructeur de la classe `Individu` a 3 paramètres qui permettent d'initialiser les attributs `_nom`, `_prenoms` et `_naissance` d'un objet représentant un individu supposé vivant. La méthode `est_mort()` renvoie `True` si un individu est décédé, et `False` sinon. L'instruction `print(i)`, où `i` est un `Individu`, provoque l'affichage de prénoms suivi de nom.

Définir la classe `Individu` en suivant les spécifications mentionnées ci-dessus. Créer des objets ou décorateurs `property` permettant l'accès aux attributs d'un individu `i` avec les instructions `i.nom`, `i.prenoms`, `i.naissance`, `i.mort`. On fera un accès en lecture uniquement pour les 3 premiers attributs. L'accès en écriture sur l'attribut `_mort()` permet d'ajouter une date de mort. Elle déclenche une exception si une date de mort existe déjà ou si celle proposée est antérieure à la date de naissance. Si l'individu est encore vivant, l'accès en lecture de sa date de mort déclenche une exception.

2.3 Définir la classe `Mariage`

Un objet de la classe `Lien` représente un lien entre deux individus `i1` et `i2` :

```
class Lien :
    def __init__(self, i1:object, i2:object) :
        if not isinstance(i1, Individu) or not isinstance(i2, Individu):
            raise TypeError("type 'dindividu invalide ")
        self._lien= (i1, i2) # packing de i1 et i2 dans un tuple

    def __str__(self) -> str :
        return f"{self._lien[0]} et {self._lien[1]}"
```

Un objet de la classe `Mariage` est un type spécial de `Lien` qui est caractérisé en plus par une date de mariage et éventuellement par une date de divorce. Le constructeur de cette classe a 3 paramètres représentant les individus et la date de mariage. La méthode `ajouter_divorce()` permet d'ajouter une date de divorce postérieure à la date de mariage. L'instruction `print(m)`, où `m` est un objet `Mariage`, provoque l'affichage de la chaîne "`Mariage entre`", suivi des deux individus, suivi de la date de mariage et éventuellement de la date de divorce. Définir la classe `Mariage`.

2.4 Définir la classe `Famille`

Un objet `Famille` est caractérisé par un attribut `_individus` de type `list` qui contient des objets `Individu` et un attribut `_mariage` de type `list` qui contient des objets `Mariage`. La méthode `ajouter_individu()` permet d'ajouter un individu à la famille. Si `f` est un objet `Famille`, l'instruction `f.ajouter_mariage(i1,i2,m)` ou `f.ajouter_mariage(i1,i2,m,d)` (avec `m`=date de mariage et `d`=date de divorce) ajoute un lien de mariage entre `i1` et `i2`. Définir la classe `Famille`.

2.5 Programme principal

« Laura Dupond » et « Paul Berger » sont deux personnes nées le 05/02/1986 et 01/07/1985 . « Paul » et « Laura » se sont mariés le 25/02/2012, ils ont eu une fille « Claire » le 03/05/2015 puis ont divorcé le 26/08/2018. Malheureusement Paul est décédé le 10/09/2024..

Écrire le main() permettant d'instancier la famille « Dupond-Berger »